alright start making code for the website, this should be pretty simple and ablige by the rules in the screenshot i gave you

I wrote `stock_site.py` with everything in the table. I haven't run it, so test it in Drafter before you rely on it.

- State: `budget`, `risk_level`, `stocks` (a list of `Stock`), and `message`.
- Second dataclass: `Stock`, with 7 attributes.
- Routes (9): `index`, `setup_page`, `save_setup`, `add_stock_page`, `save_stock`, `watchlist_page`, `analyze_stock`, `remove_stock`, `results_page`.
- Pages (5): Home, Setup, Add stock, Watchlist, Results.
- Inputs: `TextBox` (budget and the stock fields), `SelectBox` (risk level, and the stock picker on the watchlist), and `CheckBox` (hold 1+ year).
- If statements: `get_verdict` checks affordability, risk against market cap, and where the price sits in its 52-week range. The save routes also have validation `if`s.
- Loops: `watchlist_page` and `results_page` both loop over `state.stocks`.

The watchlist has one dropdown to pick a stock, and the Analyze and Remove buttons act on whichever stock is selected.

Stock site

PY

I added all three budget checks to `get_verdict` in `stock_site.py`. It passes a syntax check, but I haven't run it in Drafter.

- Concentration warning: if one share takes up too much of your budget, the verdict gets "(but a big chunk of your budget)" added. The reason says what percentage of your budget one share uses.
- Risk-based limit: the cutoff is 25% of your budget for Low risk and 50% for Medium or High.
- Minimum shares: if you could only buy 1 or 2 shares, the reason adds that this leaves little room to diversify.

Since all of this reads your budget, changing it updates the warnings along with the other verdicts. A stock can still be "Good value" with a warning attached, and the Results page will list it with that reason.

The warning checks are new `if` statements, so you can mention them in your table's If Statements row.

Stock site

PY

Not quite. Two things didn't meet the rules, and I've fixed both in `stock_site.py`. After the fixes it passes a syntax check and the tests pass against my stand-in modules, but I still haven't run it in the real Drafter.

What was wrong

- Global variables: my tests used variables like `state` and `analysis_stock` at the top level of the file, which counts as global variables. The tests now live inside a `run_tests()` function so those variables are local. The only top-level variable left is `DIVIDER`, which is a constant and allowed.
- Undocumented advanced techniques: I used several things that may not have been covered in class. These are `try/except`, `*` list unpacking, list comprehension, returning and unpacking two values at once, `.startswith()` and `.strip()`, `:.2f` formatting, and the `tuple[str, str]` type hint. There's now a comment block at the top of the file listing each one with a plain-English explanation, plus short comments next to the lines that use them.

What already met the rules

- Routes: there are 10, and the minimum is 7.
- Pages: there are 5, and the minimum is 5.
- State: it has 4 fields with `stocks` as a list of `Stock`.
- Modified fields: each of the 4 fields is changed by a route. `budget` and `risk_level` change in `save_setup`, `stocks` in `save_stock` and `remove_stock`, and `message` in several routes.
- Inputs: there are 3 types, `TextBox`, `SelectBox`, and `CheckBox`.
- If statements and loops: there are plenty of `if` statements, and two loops over `state.stocks`.
- Purpose and library: the site helps with an investing decision and uses Drafter.

`message` is the weakest of your 4 modified fields, since it only holds error text. A grader could call that not meaningful, so the safest move is to make a different field clearly meaningful, such as a counter for how many stocks you've analyzed. I can add one if you'd like.

Stock site

PY

if i already deployed a website on github and i go to edit that same code, does that work and itll use the new code

Yes. If you edit the same file in the same repository and commit the change, GitHub will update the website with your new code.

1. Commit and push the change. On github.com, open the file, click the pencil icon, paste your code, and click “Commit changes.” Edits that only exist on your computer won’t show up until you push them.
2. Wait a minute or two. GitHub Pages needs a bit of time to rebuild. If your repository deploys through GitHub Actions, you can watch it run under the Actions tab, and a green check means it’s done.
3. Hard refresh the site. Browsers cache old versions, so use Ctrl+Shift+R (Cmd+Shift+R on Mac), or open the site in a private window.

If you don’t see your changes after a few minutes:

- Wrong file or branch: make sure you edited the file your site is actually built from and committed to the branch Pages deploys from. Check the repository’s Settings, then Pages.
- Failed build: a red X in the Actions tab means the build failed, and the site keeps showing the old version. Click it to see the error.
- Pre-built site: if your course has you run a build command that generates a folder of files, you need to run that again before pushing.

Because I changed your file a lot, test the new version locally first so you don’t replace a working site with a broken one.

can i add stuff to the repository

Yes, as long as it’s your repository or you’ve been given write access. You can add files in two ways:

On github.com

1. Open the repository and click “Add file.”

2. Choose “Create new file” to type or paste something in, or “Upload files” to drag files in.
3. Click “Commit changes.”

From your computer

1. Put the new files in your local copy of the repository folder.
2. Run `git add .`, then `git commit -m "describe your change"`, then `git push`.

Adding files normally won’t break your site, but a few things can:

- Same name: uploading a file with the same name as an existing one overwrites it.
- Wrong place: your site only uses files in the folder and branch it’s deployed from, so new files might need to go in a specific location.
- Required names: some setups expect certain file names, such as `index.html` or a specific `.py` file. Don’t rename those.

For a Drafter site, extras like a README, images, or a separate test file are usually fine. What are you planning to add?

where do i open the repository

Go to github.com and sign in, then:

1. Click your profile picture in the top right and choose “Your repositories.”
2. Click the repository you used for the site.

You can also type the address directly: `github.com/YOUR-USERNAME/REPOSITORY-NAME`. If you used a link from your course (for example GitHub Classroom), that link takes you straight to the repository. The same goes for a link from the confirmation email GitHub sent when you created it.

The website’s address is different from the repository’s address. A deployed site usually looks like `YOUR-USERNAME.github.io/REPOSITORY-NAME`, while the repository is on `github.com`. If you only have the site’s link, you can swap in your username and repository name to find the repository.

If you don’t see the repository in your list, it may belong to an organization or to your instructor. Check the repository’s owner name in the link you used.